

IACMS

Inter-Agency Case Management System

A Multi-Tenant, Dynamic Workflow Case Management Platform

Full Project Documentation

Combined Design Specification and Technical Reference

Table of Contents

Table of Contents.....	2
1. Introduction.....	4
1.1 Purpose.....	4
1.2 Scope.....	4
2. System Overview.....	4
2.1 Core Concept.....	4
Case Management Layer.....	4
Workflow Definition Layer.....	4
Workflow Execution Layer.....	5
2.2 Platform Composition.....	5
3. High-Level Architecture.....	5
3.1 Core Architecture Components.....	5
3.2 Primary Runtime Dependencies.....	5
4. Repository Structure.....	6
5. Service Breakdown.....	6
6. Core Data Entities.....	7
7. Multi-Tenancy Architecture.....	7
7.1 Tenant Isolation.....	7
7.2 Recommended Approach.....	7
8. Workflow Engine.....	8
8.1 Workflow Definition (Configuration Phase).....	8
Creating a Workflow.....	8
Example Workflows.....	8
Rules.....	8
8.2 Workflow Execution Engine.....	8
Case Creation Flow.....	8
Transition Execution.....	8
Validation Rules.....	9
8.3 Workflow Versioning (Critical Feature).....	9
Problem.....	9
Solution.....	9
9. “Other Side Visibility” (Tracking Feature).....	9
9.1 What Users Should See.....	9
9.2 Example Output.....	9
10. Authentication and Access Control.....	10
10.1 Authentication.....	10
10.2 Role-Based Access Control.....	10
11. Audit and Logging.....	10
11.1 Why It Matters.....	10
11.2 What to Log.....	11
11.3 Implementation.....	11
12. Notifications.....	11
12.1 Types.....	11
12.2 Trigger.....	11
13. Data and Infrastructure.....	11
13.1 Database.....	11
13.2 Cache and Sessions.....	11
13.3 Messaging.....	11

14. API Conventions.....	12
15. Local Development Setup.....	12
15.1 Prerequisites.....	12
15.2 Start Infrastructure.....	12
15.3 Apply Database Migrations and Seed Data.....	12
15.4 Start Services.....	12
16. Useful Commands.....	12
17. Testing and Validation.....	13
18. Deployment Considerations.....	13
18.1 Scalability.....	13
18.2 Security.....	13
18.3 Performance.....	13
19. Documentation Map.....	13
20. Current Project Status.....	14
21. Risks and Challenges.....	14
22. Notes for Future Development.....	14
23. Conclusion.....	15

1. Introduction

1.1 Purpose

This document provides a comprehensive design and technical reference for IACMS, a multi-tenant case management system built around dynamic, tenant-defined workflows. It consolidates the original system design specification with the technical documentation of the implemented platform, so that product, engineering, and operations stakeholders have a single source of truth for how the system is intended to work and how it is actually built.

Each tenant (organization) using IACMS can define, manage, and execute its own workflow processes independently, while the underlying platform provides shared infrastructure for authentication, role-based access control, auditability, and event-driven integration.

1.2 Scope

The system is designed to:

- Support multiple tenants (organizations) on a single shared platform.
- Allow each tenant to define custom workflows, independent of other tenants.
- Manage cases that follow tenant-specific workflow definitions.
- Track case progress, ownership, and complete history.
- Provide visibility into “the other side” — who is responsible for a case at any point in time, and what happens next.
- Support cross-agency collaboration through referral flows.
- Provide tenant-aware data isolation, audit trails, and integration capabilities through APIs and event streams.

2. System Overview

2.1 Core Concept

Functionally, the platform is organized around three conceptual layers that together implement dynamic, configurable workflows:

Case Management Layer

Handles:

- Case creation
- Case data storage
- Case tracking

Workflow Definition Layer

Allows tenants to:

- Define steps (states)
- Define transitions (rules between steps)

Workflow Execution Layer

Responsible for:

- Moving cases between steps
- Enforcing rules
- Logging actions

2.2 Platform Composition

At the implementation level, IACMS (Inter-Agency Case Management System) realizes this concept as a multi-tenant, workflow-driven case management platform built as a Node.js microservices-based backend, designed to support government and partner agencies that need structured case handling, role-aware approvals, auditability, and event-driven integration. The platform combines:

- a central API gateway for routing and authentication,
- domain-specific services for auth, RBAC, cases, workflows, referrals, audit, integration, and notifications,
- a shared Prisma schema and shared utilities for cross-service consistency,
- PostgreSQL for transactional data, Redis for session/cache support, and Kafka for event-driven workflows.

3. High-Level Architecture

The system is organized as a modular monorepo with shared infrastructure and independently deployable services.

3.1 Core Architecture Components

- API Gateway — single entry point for clients
- Auth Service — authentication, tenant-aware login, token handling
- RBAC Service — roles and permissions
- Workflow Service — workflow definitions and publishing logic
- Case Service — case lifecycle, transitions, and assignments
- Referral Service — inter-agency case handoff
- Audit Service — audit trail ingestion and retrieval
- Integration Service — external integration foundations
- Notification Service — event-driven notifications

3.2 Primary Runtime Dependencies

- Node.js for application runtime
- Express for service APIs
- PostgreSQL for operational data
- Prisma for database access and migrations

- Redis for session and cache support
- Kafka for asynchronous event publishing
- Docker Compose for local infrastructure

4. Repository Structure

The codebase is organized as a single monorepo separating independently runnable services from shared, cross-cutting code:

```
IACMS/
├── services/                                # Microservices
│   ├── api-gateway/
│   ├── auth-service/
│   ├── rbac-service/
│   ├── case-service/
│   ├── workflow-service/
│   ├── referral-service/
│   ├── audit-service/
│   ├── integration-service/
│   └── notification-service/
├── shared/                                # Cross-service shared code
│   ├── common/
│   ├── middleware/
│   ├── contracts/
│   └── utils/
├── prisma/                                # Prisma schema and migrations
├── infrastructure/                        # Docker and local environment config
├── docs/                                  # Additional documentation
├── scripts/                              # Setup and operational scripts
└── package.json                          # Root workspace configuration
```

5. Service Breakdown

Each service is independently runnable and owns a distinct responsibility within the platform:

Service	Port	Responsibility
API Gateway	3000	Main public entry point, auth checks, routing, RBAC enforcement
Auth Service	3001	User login, registration, JWT/session handling
RBAC Service	3002	Roles, permissions, user-role assignments
Case Service	3003	Case lifecycle, state transitions, assignment logic
Workflow Service	3004	Workflow definitions, publication, versioning
Referral Service	3005	Cross-tenant and cross-agency referrals

Service	Port	Responsibility
Audit Service	3006	Audit event ingestion and retrieval
Integration Service	3007	External integrations and webhook support
Notification Service	3008	Event-driven notifications

6. Core Data Entities

The shared Prisma schema models the following core entities, which together support tenant-aware, workflow-driven case handling:

Entity	Description
Tenant	Represents an organization using the platform.
User	Represents a system user belonging to a tenant.
Role	Defines user responsibilities and permission scope.
Workflow	Represents a workflow definition owned by a tenant.
Workflow Step (State)	Represents a single stage in a workflow.
Workflow Transition	Defines an allowed movement between two steps.
Case	Represents a business record progressing through a workflow.
Case History (critical)	Tracks every transition a case has gone through, including who acted, when, and the resulting step change.

7. Multi-Tenancy Architecture

7.1 Tenant Isolation

Each tenant must have:

- Separate workflows
- Separate users and roles
- Separate case data

7.2 Recommended Approach

- Use a `tenant_id` column in all major tables.
- Enforce tenant filtering at the application level, in addition to schema-level constraints.

Each tenant operates in an isolated scope, and the platform is designed to support tenant-aware reads, writes, and workflow behavior throughout every service. Referral flows are also

included to allow cross-agency collaboration when needed, without breaking tenant boundaries for ordinary case data.

8. Workflow Engine

Workflows define the steps a case can occupy and the transitions allowed between them. Cases are created from published workflows and progress through allowed state transitions based on workflow configuration and role-based rules.

8.1 Workflow Definition (Configuration Phase)

Creating a Workflow

Steps:

1. Tenant creates a workflow
2. Tenant adds steps
3. Tenant defines transitions between steps
4. Tenant assigns roles to steps and transitions

Example Workflows

Tenant A:

- Draft → Review → Approval → Closed

Tenant B:

- Open → Investigate → Escalate → Resolve → Close

Rules

Each workflow must have:

- One initial step
- At least one final step
- Only valid transitions — no broken links

8.2 Workflow Execution Engine

Case Creation Flow

5. User selects a workflow
6. System assigns the initial step and the workflow version
7. Case is created

Transition Execution

When a user performs an action, the system follows this step-by-step process:

8. Fetch the current case step
9. Validate that the action is allowed

10. Validate that the user has the required permission
11. Find the matching transition
12. Update the case: set the new step
13. Log the transition to case history
14. Trigger notifications

Validation Rules

- User must have the required role for the action.
- The transition must exist in the workflow definition.
- The case must not already be closed.

8.3 Workflow Versioning (Critical Feature)

Problem

Workflow definitions can change over time. The system must account for what happens if a workflow changes while cases are actively in progress under the previous definition.

Solution

- Assign a version number to each published workflow.
- Cases store the workflow version they were created under.
- Existing cases continue executing against their original workflow version, even after a newer version is published.

Within IACMS, the Workflow Service owns workflow definitions, publication, and versioning, ensuring that in-flight cases remain consistent regardless of later changes to a tenant's workflow configuration.

9. “Other Side Visibility” (Tracking Feature)

9.1 What Users Should See

For each case, users should be able to see:

- Current step
- Responsible role or user
- Next possible actions
- Previous actions (full history)

9.2 Example Output

Field	Value
Case	Investigation Request

Field	Value
Current Step	Review
Responsible	Officer
Available Actions	Approve → Supervisor Reject → Applicant

This feature is what differentiates IACMS from a simple case tracker: at any point, any authorized participant can see exactly where a case stands, who currently owns it, and what the possible next steps are — without needing to ask another party directly.

10. Authentication and Access Control

10.1 Authentication

The system supports both:

- Session-based authentication for browser-style clients.
- JWT-based authentication for APIs and integrations.

The API Gateway validates identity and forwards tenant and role context to downstream services, so individual services do not need to re-implement authentication logic.

10.2 Role-Based Access Control

Permissions are enforced at:

- Step level
- Transition level

Governing rules:

- Only assigned roles can act on a given step.
- Only allowed roles can trigger a given transition.

The RBAC Service owns roles, permissions, and user-role assignments, while enforcement of those rules at request time is shared between the API Gateway and the Case and Workflow services.

11. Audit and Logging

11.1 Why It Matters

- Accountability
- Debugging
- Compliance

11.2 What to Log

- Who performed the action
- When the action occurred
- From → To step
- Comments associated with the action

11.3 Implementation

The platform publishes structured events for important actions. Audit consumption is handled by a dedicated Audit Service, decoupled from the primary business services via Kafka, so that compliance logging remains reliable even under load on other parts of the system.

12. Notifications

12.1 Types

- In-app (WebSocket)
- Email

Note: the current notification implementation should be maintained as-is rather than redesigned as part of this workflow effort.

12.2 Trigger

- Notifications are triggered on step change.

13. Data and Infrastructure

13.1 Database

PostgreSQL is the primary transactional store. The shared Prisma schema defines the models and relationships used across the microservices.

13.2 Cache and Sessions

Redis supports:

- Session storage for gateway authentication flows
- Fast cached access for frequently used lookups
- Stateful web interactions

13.3 Messaging

Kafka provides event-driven communication between services. Events are defined and shared through the repository's contract and utility layers, allowing actions such as audit logging, notifications, and workflow-related events to be handled independently and reliably.

14. API Conventions

The project exposes APIs through the API Gateway on port 3000. Requests are routed to the appropriate backend service based on route patterns.

Concerns handled centrally by the gateway include:

- Authentication
- Tenant context
- Role-based access enforcement
- Request forwarding to downstream services

15. Local Development Setup

15.1 Prerequisites

- Docker Desktop or Docker Engine
- Node.js 20+
- npm
- Postman (recommended for API testing)

15.2 Start Infrastructure

```
cd infrastructure
docker-compose up -d postgres redis kafka zookeeper
```

15.3 Apply Database Migrations and Seed Data

```
cd ..
npx prisma migrate deploy
node prisma/seed.js
```

15.4 Start Services

Each service can be started independently, for example:

```
cd services/auth-service
npm start
```

...and similarly for the other services as needed.

16. Useful Commands

From the repository root:

```
npm run docker:up
npm run docker:down
npm run prisma:migrate
npm run prisma:studio
npm run db:seed
npm run test:gateway
```

17. Testing and Validation

The repository includes service-specific test folders and supporting tooling. Validation typically includes:

- Unit and integration tests within each service folder
- API exercise through Postman collections
- Infrastructure checks via Docker Compose
- Database migration validation

The repository also includes Postman collections for authentication and platform API testing.

18. Deployment Considerations

18.1 Scalability

Use indexing on:

- tenant_id
- case_id

18.2 Security

- Enforce tenant isolation strictly across every service and query.

18.3 Performance

- Cache workflow definitions to avoid repeated lookups on every transition.

19. Documentation Map

The repository contains several focused documents in addition to this consolidated reference:

Document	Contents
README.md	Project overview and quick start
docs/	Architectural explanation

Document	Contents
SYSTEM_OVERVIEW.md	
docs/ DOCUMENTATION.md	Technical reference material
docs/ KAFKA_INTEGRATION.md	Event bus details
docs/ TENANT_ISOLATION.md	Multi-tenancy design notes
docs/ SESSION_AUTHENTICATI ON.md	Session authentication flow
PHASES.md	Roadmap and phase planning

This document serves as a consolidated entry point spanning both the original design intent and the current technical implementation of the full project.

20. Current Project Status

The codebase is a working multi-service platform with workflow, case, authentication, and audit capabilities already implemented. The project is structured for extension and further hardening, including additional integration and operational improvements.

21. Risks and Challenges

- Over-engineering workflows beyond what tenants actually need.
- Poor UI/UX for workflow creation, which can undermine an otherwise solid engine.
- Data inconsistency if transition and permission validation is weak or inconsistently enforced.

22. Notes for Future Development

Recommended next steps include:

- Complete end-to-end validation across all services.
- Expand automated test coverage.
- Tighten observability and deployment pipelines.
- Refine production-grade security, resilience, and monitoring.

23. Conclusion

IACMS is, at its core, a workflow-driven platform rather than a simple case tracker. It combines a configurable workflow engine — with versioned, tenant-specific definitions, role-based step and transition permissions, and full “other side” visibility — with a modern, multi-tenant microservices architecture covering authentication, RBAC, case management, referrals, auditing, integration, and notifications.

Success depends on three things holding together over time:

- Strong workflow engine design — correct, predictable transition and versioning behavior.
- A clear data model — explicit tenant, workflow, step, transition, case, and history entities.
- Strict validation and audit logging — so every action is permissioned, traceable, and compliant.

Together, these principles support structured case handling, secure tenant isolation, role-based access, and audit-friendly operations at scale.